

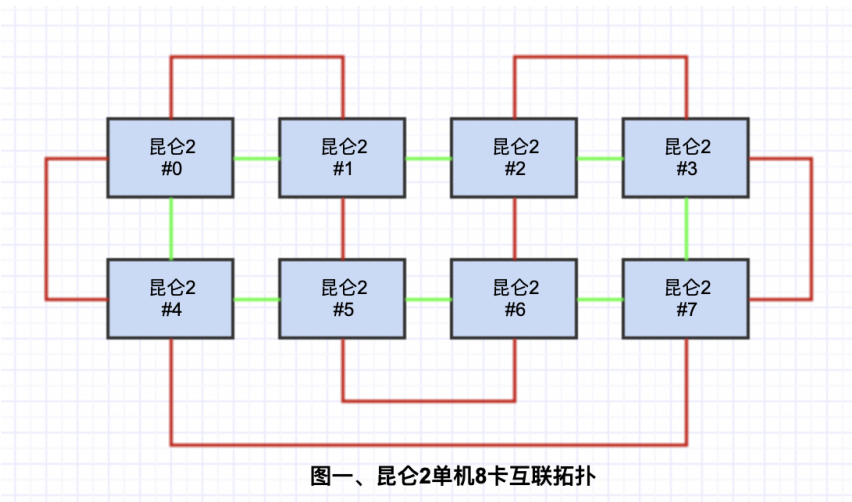
昆仑2通讯库设计文档

 本页面由边俊柯于2022/06/17迁移自wiki 陈庆澍空间

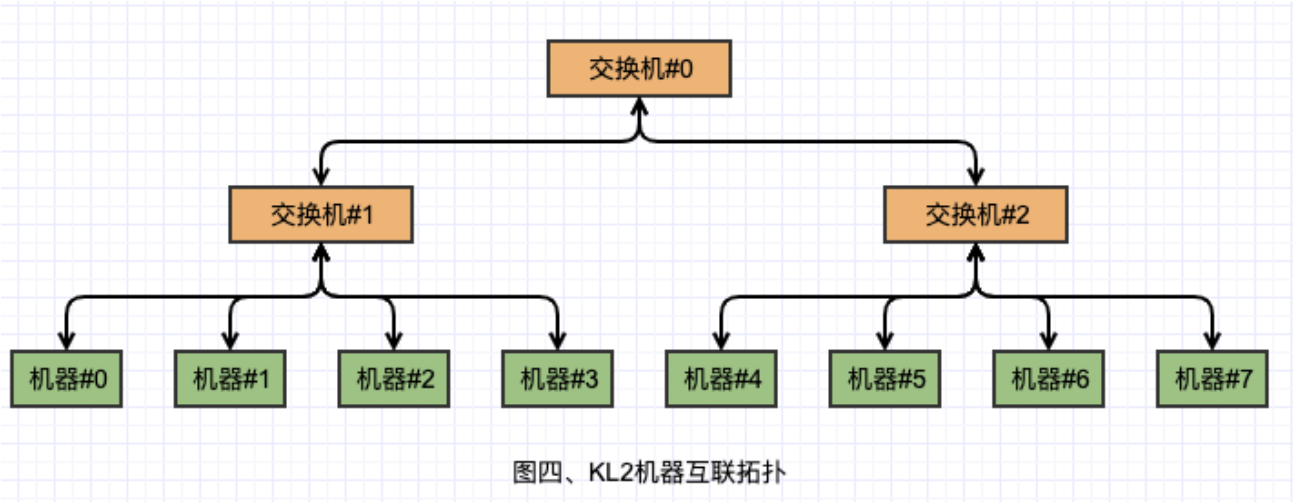
一、概述

昆仑2是一款支持训练+推理的AI芯片，是昆仑系列第二款AI芯片。相比于昆仑1，昆仑2提供了更高算力、更大内存，且增加了芯片间的高速互联。本文档基于昆仑2的特性，详细的描述了昆仑2通讯库的设计方案。

二、硬件互联



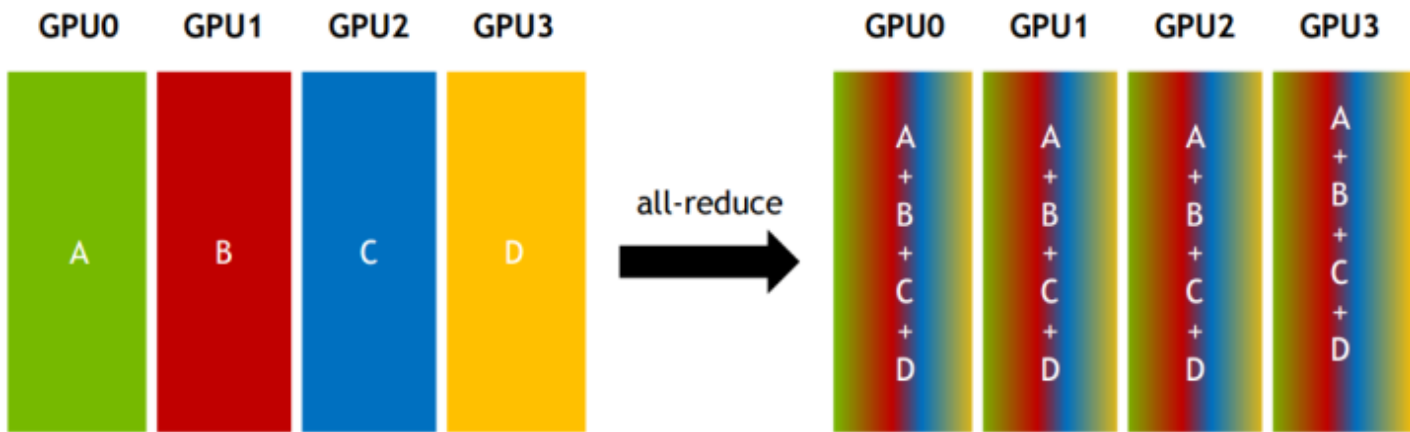
如图一所示，每张昆仑2上有4个CCIX口，每个口提供双向50GB的带宽，单张芯片提供双向200GB的带宽。单机内部8卡可以通过组成两个双向ring进行数据交互（图中红色一个双向ring，绿色一个双向ring）。



多节点（机器）之间使用100Gb的RDMA网卡，通过交换机相连。

三、通讯算法

昆仑2节点内部通讯主要使用CCIX，单机8卡可以组成4个CCIX单向ring。多机之间通过RDMA进行通讯。



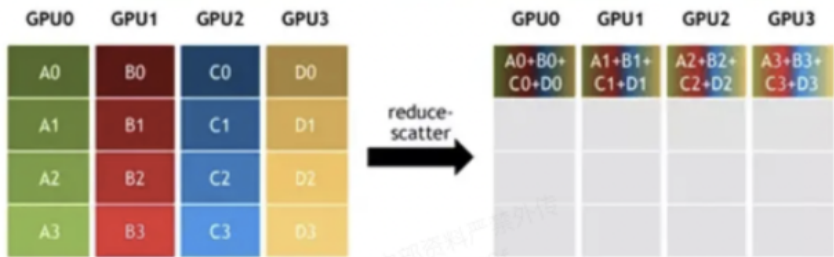
由于CCIX与RDMA的带宽和发起数据传输的接口不一致，我们分三个step实现一次all-reduce的操作：

- step 1：单机内部调用reduce scatter操作，节点内部每张卡有部分节点内完整数据
- step 2：多节点之间对每张卡拥有的部分节点完整数据调用all reduce
- step 3: 单机内部调用broadcast

3.1 节点内reduce scatter

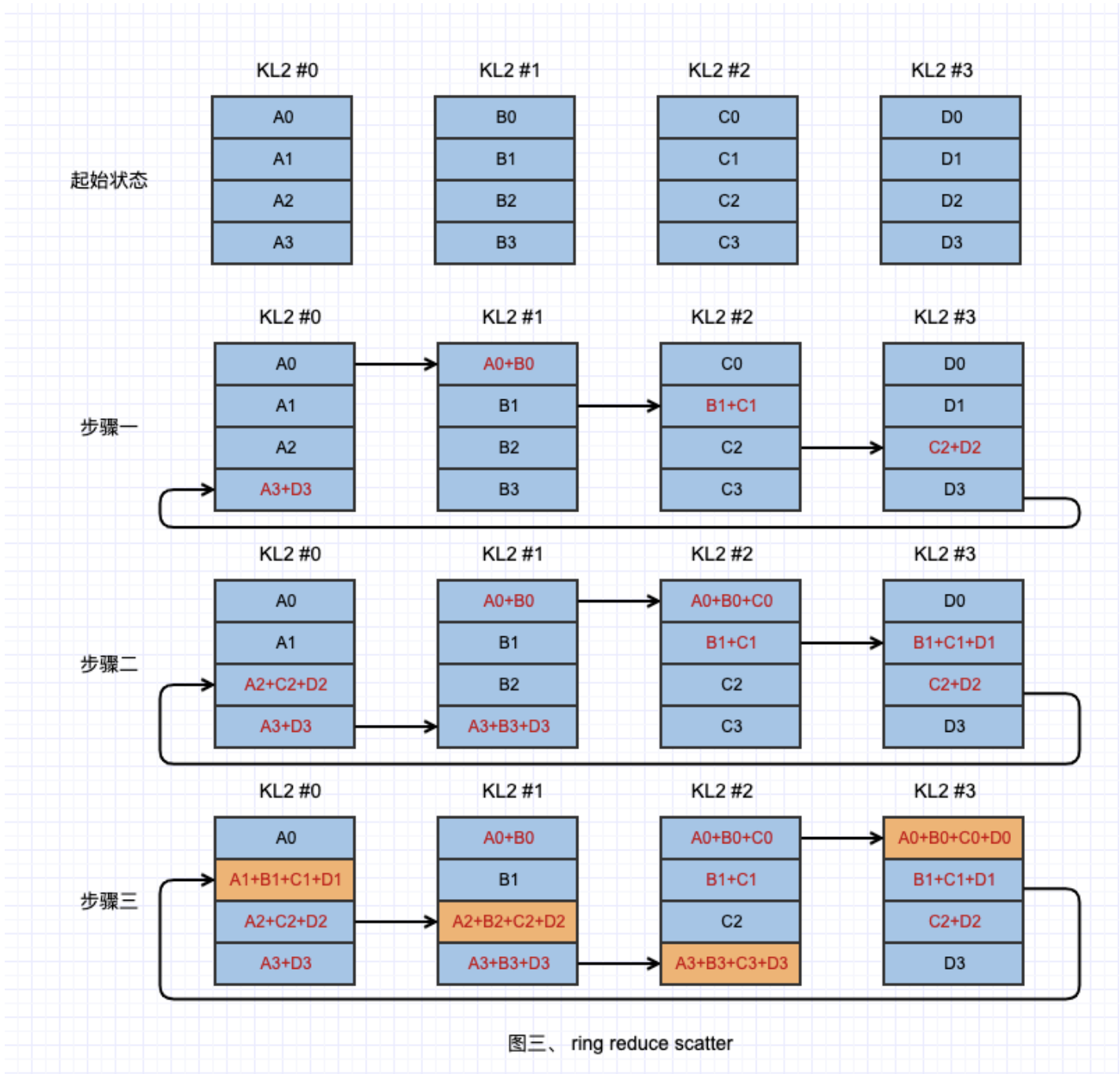
REDUCE-SCATTER

Combine data from all senders; distribute result across participants



图二、4张GPU reduce scatter例子

图二展示了reduce scatter的语义，以4张卡为例，通过reduce scatter操作，每张卡都拥有了1/4完整的数据。



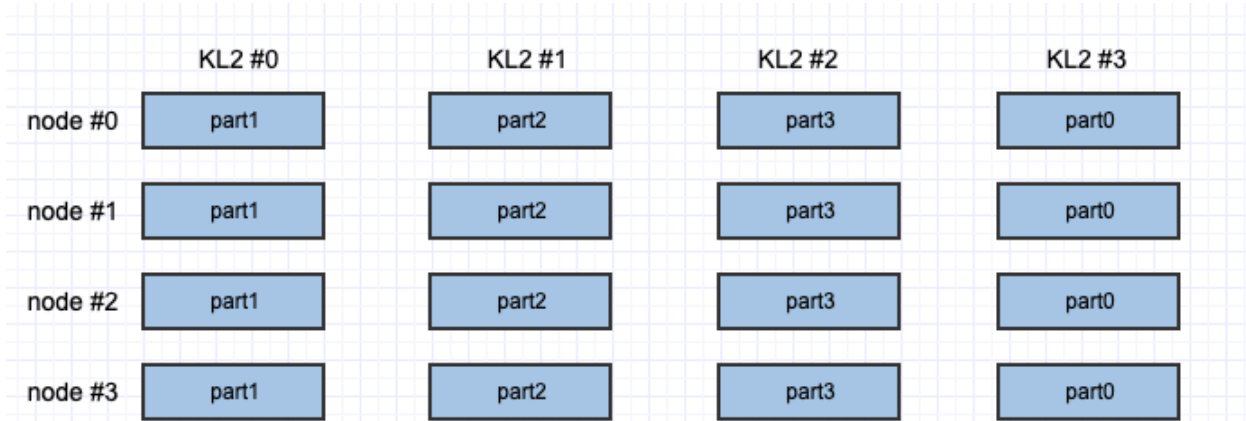
昆仑2内部CCIX互联采用了ring结构，所以内部reduce scatter采用ring reduce scatter。图三展示了4张卡ring reduce scatter的过程。4张KL2的情况下，首先将需要reduce-scatter的数据均分为4份，如KL2 #0的A0, A1, A2, A3, 在每个步骤中，将自己的1/4数据发给下一张卡，下一张卡接受到数据后，与本地的数据累加，比如步骤一种KL2 #1收到了KL2 #0的A0数据，与本地的B0累加，得到A0+B0。下一个步骤为将累加后的数据发给下一张卡，比如KL2 #1把A0+B0的结果发给KL2 #2, KL2 #2收到后与本地的C0累加得到A0+B0+C0, 重复上述步骤n-1次（n为单节点卡的数量），每张卡都有1/n累加后的结果。比如完成ring reduce scatter后，KL2 #0有A1+B1+C1+D1, KL2 #1有A2+B2+C2+D2等。

ring reduce scatter开销分析

假设有m个节点，每个节点n张卡，每张卡上需要all reduce的数据总量为B，假设节点内CCIX带宽是X，累加的速度是A。reduce scatter总共需要n - 1个步骤，每个步骤需要一次发送数据+一次累加，耗时是B/n/X+B/n/A, 所以完成reduce scatter的耗时是(n-1)*B/n/X + (n - 1) * B/n/A = B * (1 - 1/n) * (1/X + 1/A)

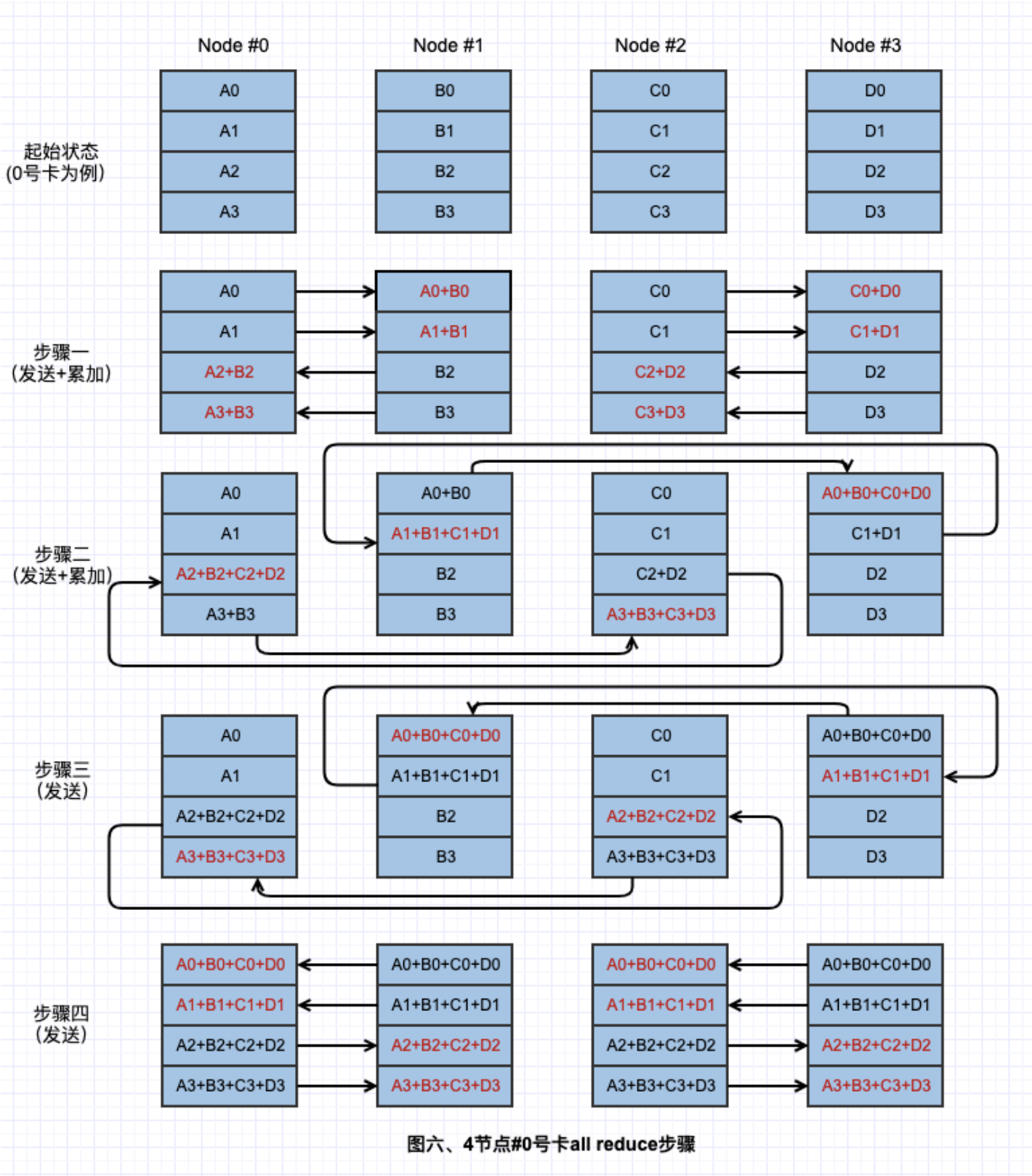
3.2 节点（机器）间all reduce

节点间通过RDMA网卡+以太网交换机相连，支持任意两个机器间P2P通讯。为了避免ring all reduce算法因为节点增加，延时开销会线性增加，机器间的all reduce采用double binary tree的通讯算法。



图五、4节点reduce scatter后状态

图五展示了4个节点，每个节点4张卡，经过step 1 reduce scatter之后，每张KL2包含的数据,比如每个节点#0 KL2包含了part1的数据（A1+B1+C1+D1）。step 2的内容是进行节点间的all reduce。所有节点对应卡号的KL2进行all reduce, 比如所有节点的#0 KL2进行all reduce, all reduce后的数据是每个节点的#0 KL2都有4个节点part1数据累加后的结果。

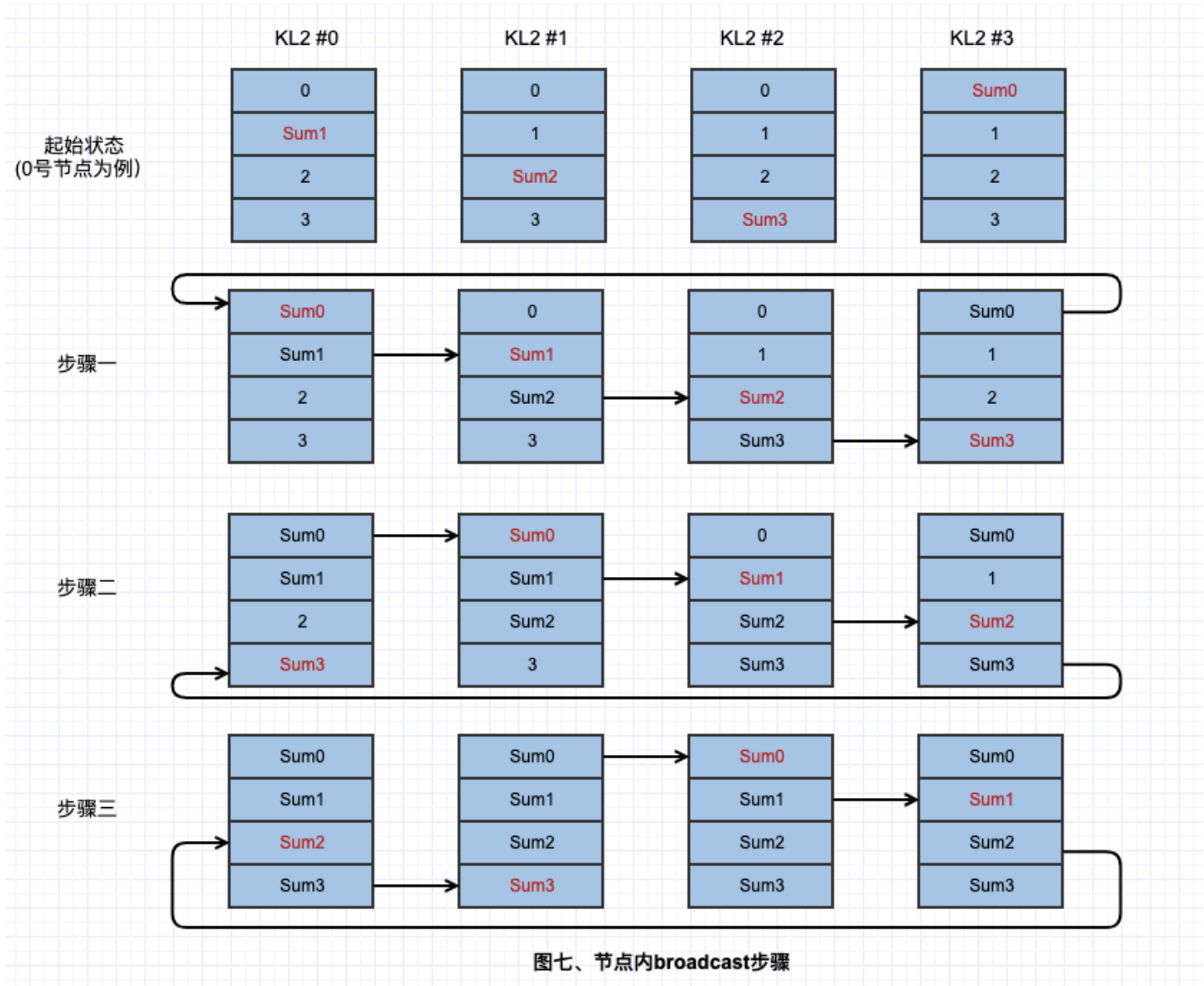


图六展示了4个节点相应卡进行all reduce的步骤。首先，node 0和node 1配对，node 2和node 3配对，node 0将前半部分数据通过RDMA网卡发给node 1，node 1将后半部分数据通过RDMA网卡发送给node 0，node 2和node 3也进行同样的步骤，且每个node接受到数据后都与本地数据进行累加。第二步时，node 0和node 2配对，node 1和node 3配对，每个node将之前累加后的数据的一半发给对方，收到数据后，再与本地数据进行累加。该步骤后，每张卡都有1/4完整的数据。之后按照与开始配对相反的顺序进行配对，然后发送数据。比如步骤一是node 0与node 1配对，步骤二是node 0和node 3配对，那之后两步就是步骤三是node 0和node 3配对，步骤四是node 0和node 1配对。步骤三和四均是将自己节点的完整数据发送给配对方。

节点间all reduce开销分析

节点间all reduce使用double binary tree的all reduce算法，假设有m个节点，每个节点n张卡，数据总量是B，节点间RDMA的带宽是Y，累加的速度是A。需要发送的数据总量是 $2*B*(1-1/m)$ ，需要累加的数据总量是 $B*(1 - 1/m)$ ，总的开销为 $2*B*(1 - 1/m)/Y + B*(1 - 1/m)/A = B*(1-1/m)*(2/Y + 1/A)$

3.3 节点内all gather



经过step 1 reduce scatter和step 2 all reduce之后，每张KL2卡上都包含了一部分所有卡累加的结果，如图7中初始状态的Sum。step 3需要调用broadcast将部分累加后的结果发送给节点内所有的KL2。broadcast采用ring broadcast的算法，每次都把自己的数据发送给ring中的下一个卡，经过n-1个步骤后，每张KL2都有完整的累计后的数据。

节点内broadcast开销分析

节点内broadcast使用ring broadcast，假设节点内有n张卡，all reduce总数据量是B，节点内的带宽是X，则总开销是 $B*(1-1/n)/X$

3.4 all reduce开销分析

一个完整的all reduce包含了上述的3个step， 假设总共有m个节点， 每个节点n张KL2， all reduce的数据量是B， 节点内的带宽是X， 节点间是Y， 累加的速率是A， 则总开销是： $B * (1 - 1/n) * (1/X + 1/A) + B*(1-1/m)*(2/Y + 1/A) + B*(1-1/n)/X = B * (1 - 1/n) * (2/X + 1/A) + B * (1 - 1/m) * (2/Y + 1/A)$

当n和m无限大时， 开销为 $2B * (1/X + 1/Y + 1/A)$

多机场景下， 每个节点8个KL2， 带宽X=100GB/s, Y=100Gb/s=12.5GB/s, A=2400GFLOPs, 下表列出m=2, 4, 8, 16, 256时， 各部分耗时

	m	CCIX	RDMA	累加(上限,m,n无限大)	CCIX:RDMA:累加
2	2	7B/400	2B/25	B/4800	84： 384： 1
3	4	7B/400	3B/25	B/4800	84： 576： 1
4	8	7B/400	7B/50	B/4800	84： 672： 1
5	16	7B/400	3B/20	B/4800	84： 720： 1
6	256	7B/400	51B/320	B/4800	84： 765： 1

由表格可知， **多机通讯的瓶颈在RDMA网卡的带宽。**

四、其他

4.1 KL1 & KL2兼容

通讯库同时支持KL1和KL2， KL1和KL2的接口保持一致。只需要在初始化的时候指定KL1或者KL2就可以。后期如果runtime支持检测卡类型， 初始化时就不需要手动指定。

4.2 拓扑检测

KL2节点内CCIX组成四个单向ring（如图一所示）， 需要通讯库检测卡与卡直接是否有CCIX互联， 并且依据这些信息组成四个单向ring， 具体检测方法等后期接口明确后再定义详细方案， 初期可以通过手动配置。

KL2节点间通过RDMA网卡+以太网交换机相连（如图四所示）， 不同节点之间通讯需要跨越的交换机数量不同， 比如KL2 #0和KL2 #1只需要跨一个交换机， 但是KL2 #0和KL2 #7需要跨3个交换机。通讯库需要检测节点与节点间网络距离， 并以此来确定double binary tree的配对情况。第一版通讯库可以通过手动指定节点间网络距离， 后期再加入动态检测逻辑。

4.3 压缩功能

KL2 CCIX提供压缩功能， 数据传输过程中， 可以将float32压缩为float20进行传输， 在损失一点精度的前提下， 大幅提升性能。该功能在通讯库初始化的时候进行配置， 配置完成后， 所有的通讯功能均开启压缩功能。

4.4 异常检测&恢复

链路异常时一般会触发kernel的timeout（kernel轮询flag超时），此时程序会退出，需要手动恢复链路。为实现这个功能，需要额外开发一套工具，在超时之后检测哪些链路异常，并且提供恢复链路的接口。

五、详细设计

5.1 kernel接口设计

KL2单机2卡、4卡、8卡、多机多卡等不同场景的连接拓扑不同，比如单机两卡存在一个单向ring或者两个单向ring，单机8卡存在4个单向ring，多机多卡需要RDMA通讯等。因此，在接口设计时需要充分考虑不同的网络拓扑，让接口在不同的网络拓扑中都能适用。

</> C++ | 收起 ^

```
1  /*
2   *      Introduce: redcue scatter kernel，实现的功能如3.1章节所示
3   *      -----
4   *      Input:
5   *          float* src_addr:      reduce scatter source数据地址
6   *          char* buf:            buffer地址（可以是L3或者DDR），char*数组，大小为
available_ccix_num,
7   *                                数组每一项是对应CCIX ring的buffer
8   *          char* next_buf:       next node buffer地址（可以是L3或者DDR），char*
数组，大小为available_ccix_num,
9   *                                数组每一项是对应CCIX ring的next_buf，该
next_buf已经加上了CCIX对应port的偏移
10  *          int buf_len:          每一个buffer的大小（byte）
11  *          int* pre_flag:        同步flag的在前一张卡的地址，syscon的地址数组，
12  *                                数组大小与available_ccix_num相同，pre_flag已
经加上CCIX对应port的偏移
13  *          int* cur_ready_flag:   同步flag的在当前张卡的地址，表示当前卡的buffer是
否可以被写，syscon的地址数组
14  *                                数组大小与available_ccix_num相同，数组中对应索
引的flag给相应的CCIX ring使用
15  *          int* cur_done_flag:    同步flag的在当前张卡的地址，表示前一张卡的数据是
否写到这里，syscon的地址数组
16  *                                数组大小与available_ccix_num相同，数组中对应索
引的flag给相应的CCIX ring使用
17  *          int* post_flag:       同步flag的在后一张卡的地址，syscon的地址数组
18  *                                数组大小与available_ccix_num相同，post_flag已
经加上CCIX对应port的偏移
19  *          int len:              reduce scatter float的个数
```



```

20 *          int node_num:          ring中节点总数
21 *    int available_ccix_num:      可用的CCIX ring数量, > 0 and <= 4
22 *
23 *    Output:
24 *          float* dst_addr:      reduce scatter结果存放地址
25 *
26 *    Others:
27 *          buf, next_buf, pre_flag, cur_ready_flag, cur_done_flag,
post_flag均是指向设备中的内存, 该内存
28 *          是一个大小为available_ccix_num的数组, 数组中每一项均是指针, 指向buffer或
者flag。使用该接口之前, 需要
29 *          根据ccix ring的数量, 以及ring中使用CCIX的port num, 填充相应的值。比如,
有两个ring, 则next_buf指向一
30 *          个包含了两个地址的KL2内存数组, 假设ring 0的下一张卡的buffer地址是0xa, 且
下一张卡的port是1, 则
31 *          next_buf[0] = 0x5000000000a(port 1的便宜是0x5000000000), ring 1的下
一张卡的buffer是0xf000, 且
32 *          通过port 2与下一张卡相连, 则next_buf[1]的填充内容是0x600000f000。其他几
个数据根据这个示例相应设置参数
33 *    -----
34 *
35 */
36 __global__ void ring_reduce_scatter(char* src_addr, char* dst_addr, char*
buf, char* next_buf,
37     char* pre_flag, char* cur_ready_flag, char* cur_done_flag,
38     char* post_flag, int len, int node_num, int available_ccix_num);
39
40
41
42
43 /*
44 *    Introduce: ring broadcast kernel, 实现的功能如3.3章节所示
45 *    -----
46 *
47 *    Input:
48 *          char* src_addr:          ring broadcast source数据地址, 对于非root节
点, 该值为0
49 *          char* buf:              buffer地址 (可以是L3或者DDR), char*数组, 大小为
available_ccix_num,
50 *          char* next_buf:          next node buffer地址 (可以是L3或者DDR), char*
数组, 大小为available_ccix_num,

```

```

51  *      数组每一项是对应CCIX ring的next_buf, 该
      next_buf已经加上了CCIX对应port的偏移
52  *      int buf_len:      每一个buffer的大小 (byte)
53  *      int* pre_flag:    同步flag的在前一张卡的地址, syscon的地址数组,
54  *      数组大小与available_ccix_num相同, pre_flag已
      经加上CCIX对应port的偏移
55  *      int* cur_ready_flag: 同步flag的在当前张卡的地址, 表示当前卡的buffer是
      否可以被写, syscon的地址数组
56  *      数组大小与available_ccix_num相同, 数组中对应索引
      的flag给相应的CCIX ring使用
57  *      int* cur_done_flag:  同步flag的在当前张卡的地址, 表示前一张卡的数据是
      否写到这里, syscon的地址数组
58  *      数组大小与available_ccix_num相同, 数组中对应索引
      的flag给相应的CCIX ring使用
59  *      int* post_flag:     同步flag的在后一张卡的地址, syscon的地址数组
60  *      数组大小与available_ccix_num相同, post_flag已
      经加上CCIX对应port的偏移
61  *      int len:           broadcast的长度 (byte)
62  *      int node_num:      ring中节点总数
63  *      int available_ccix_num: 可用的CCIX ring数量, > 0 and <= 4
64  *
65  *      Output:
66  *      char* dst_addr:     ring broadcast结果存放地址
67  *
68  *      Others:
69  *      buf, next_buf, pre_flag, cur_ready_flag, cur_done_flag,
      post_flag均是指向设备中的内存, 该内存
70  *      是一个大小为available_ccix_num的数组, 数组中每一项均是指针, 指向buffer或
      者flag。使用该接口之前, 需要
71  *      根据ccix ring的数量, 以及ring中使用CCIX的port num, 填充相应的值。比如,
      有两个ring, 则next_buf指向一
72  *      个包含了两个地址的KL2内存数组, 假设ring 0的下一张卡的buffer地址是0xa, 且
      下一张卡的port是1, 则
73  *      next_buf[0] = 0x500000000a(port 1的地址是0x5000000000), ring 1的下一
      张卡的buffer是0xf000, 且
74  *      通过port 2与下一张卡相连, 则next_buf[1]的填充内容是0x600000f000。其他几
      个数据根据这个示例相应设置参数
75  *      -----
76  *
77  */
78 __global__ void ring_broadcast(char* src_addr, char* dst_addr, char* buf,
      char* next_buf,
79      char* pre_flag, char* cur_ready_flag, char* cur_done_flag,

```

```
80      char* post_flag, int len, int node_num, int available_ccix_num);
```